

Motion Detection Application

Development Document

Python | OpenCV | Version 1.0

1. Project Summary

This document records how the Motion Detection Application was planned, built, tested, and prepared for use. It complements the Architecture Document: the architecture explains how the finished system is structured, while this document explains how it was developed, how to set it up, how each part of the code works, and how it can be maintained and extended.

Item	Description
Project name	Motion Detection Application
Goal	Detect movement from a webcam, show a message, record video, and show black-and-white video when there is no motion.
Language	Python 3.8+
Main library	OpenCV (opencv-python)
Deliverable	A single script, motion_detector.py, plus the recordings it produces
Input	Live webcam stream
Output	On-screen display, popup alerts, .avi video files, final summary alert

2. Objectives and Deliverables

2.1 Objectives

- Build a real-time motion detector that works on an ordinary laptop with no special hardware.
- Make the system state obvious to the user: black and white when idle, color with markers when motion is present.
- Capture motion events automatically so that nothing needs to be started or stopped by hand.
- Keep the code short and readable enough to be explained in a classroom or viva.

2.2 Deliverables

Deliverable	Purpose
motion_detector.py	The complete application
recordings/ folder	Video files created automatically at run time
Architecture Document	Explains the structure and design decisions
Development Document	This document: process, setup, code walkthrough, testing, maintenance

3. Development Approach

An incremental (iterative) approach was used. Instead of writing the whole program at once, the project was built in small working steps, and each step was run and checked before the next one was added. This approach suits computer-vision work because results can only be judged by looking at the live video, and problems such as false detections are found early rather than at the end.

3.1 Development Phases

#	Phase	Work done	Result
1	Requirements	Listed what the program must do: detect motion, show a message, record, use black and white when idle, show a final alert.	Seven functional requirements
2	Research and design	Compared detection methods and chose frame differencing; designed the pipeline.	Pipeline design
3	Camera and display	Opened the webcam and showed live frames in a window with a quit key.	Working live view
4	Motion detection	Added grayscale, blur, absdiff, threshold, dilate, and contour filtering.	Boxes appear on movement
5	Modes and alerts	Added black-and-white idle mode, color alert mode, banner, and threaded popup.	Two display modes
6	Recording and summary	Added automatic video recording, cooldown, event counting, and final alert.	Complete application
7	Testing and tuning	Ran test scenarios and adjusted threshold values.	Stable, tuned program
8	Documentation	Wrote the architecture and development documents.	This document set

3.2 Why Frame Differencing Was Chosen

Method	Advantages	Disadvantages
Frame differencing (chosen)	Very simple, fast, needs no training, easy to explain	Sensitive to sudden lighting change
Background subtraction (MOG2)	Adapts to gradual lighting changes	More parameters; slower to tune
Optical flow	Gives direction and speed of motion	Heavier computation
Deep-learning detector	Recognises what the object is	Needs a model and more computing power

Frame differencing gives the best balance of simplicity and results for a project whose main goal is to detect that something moved. The other methods are listed as future improvements.

4. Development Environment

4.1 Software Requirements

Component	Requirement
Operating system	Windows 10/11, macOS, or Linux
Python	Version 3.8 or newer
Libraries	opencv-python (installed with pip); tkinter (normally included with Python)
Editor	Any editor, for example VS Code, PyCharm, or IDLE
Hardware	A webcam (built-in or USB) and a standard CPU; no GPU needed

4.2 Installation Steps

1. **Install Python.** Install Python from python.org and tick "Add Python to PATH" during installation.
2. **Open a terminal.** Open a terminal or command prompt in the project folder.

3. **Create a virtual environment.** Optionally create a virtual environment so the project libraries stay separate from other projects.
4. **Install OpenCV.** Run the install command shown below.
5. **Run the program.** Run the program with the command shown below. A window opens and the camera starts.

```
python -m venv venv
venv\Scripts\activate          # Windows   (macOS/Linux: source venv/bin/activate)
pip install opencv-python
python motion_detector.py
```

4.3 Common Setup Problems

Problem	Fix
"Could not open the camera"	Close other apps using the camera, or change CAMERA_INDEX to 1.
No module named cv2	Run pip install opencv-python in the same environment you run Python from.
No module named tkinter	Linux: install python3-tk. Alerts will still print to the console without it.
Camera permission denied (macOS/Windows)	Allow camera access for the terminal or editor in system privacy settings.

5. Code Structure and Walkthrough

The program is a single file with a clear order: imports, settings, helper functions, a main function containing the processing loop, and an entry point. The sections below explain each part.

5.1 File Layout

Section	Contents
Imports	os, sys, threading, time, datetime, and cv2
Settings	CAMERA_INDEX, MIN_AREA, THRESHOLD, STOP_RECORD_AFTER, ALERT_COOLDOWN, OUTPUT_DIR
show_popup()	Non-blocking motion alert using a daemon thread
show_final_alert()	Blocking "Project Completed" alert and console summary
new_writer()	Creates a new timestamped video file and VideoWriter
main()	Camera setup, main loop, shutdown, and final summary
Entry point	if __name__ == "__main__": main()

5.2 Settings Block

All adjustable values are named constants at the top of the file. Changing behaviour therefore never requires editing the processing logic.

```
CAMERA_INDEX = 0          # 0 = default webcam
MIN_AREA = 1500           # smaller moving areas are ignored
THRESHOLD = 25            # pixel-difference sensitivity
STOP_RECORD_AFTER = 3.0   # seconds of stillness before recording stops
ALERT_COOLDOWN = 10.0     # min seconds between popups
OUTPUT_DIR = "recordings"
```

5.3 Motion Detection Core

This is the heart of the program. Each frame is converted to grayscale and blurred, then compared with the previous frame. The result is cleaned up and turned into a list of bounding boxes. If the list is not empty, motion has occurred.

```
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
gray_blur = cv2.GaussianBlur(gray, (21, 21), 0)

diff = cv2.absdiff(prev_gray, gray_blur)
_, thresh = cv2.threshold(diff, THRESHOLD, 255, cv2.THRESH_BINARY)
thresh = cv2.dilate(thresh, None, iterations=2)
contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL,
                               cv2.CHAIN_APPROX_SIMPLE)

prev_gray = gray_blur

boxes = [cv2.boundingRect(c) for c in contours
         if cv2.contourArea(c) >= MIN_AREA]
motion = len(boxes) > 0
```

The Gaussian blur removes fine noise, absdiff measures the change, the threshold makes the change binary, dilation joins nearby fragments, and the area filter removes tiny disturbances. The variable motion is the single decision that drives everything that follows.

5.4 Black-and-White and Color Modes

When there is motion, a copy of the original color frame is used and boxes and a banner are drawn on it. When there is no motion, the gray image is converted back to three channels so that it can still be written to the video file.

```
if motion:
    display = frame.copy()                # color
    for (x, y, w, h) in boxes:
        cv2.rectangle(display, (x, y), (x+w, y+h), (0, 255, 0), 2)
    cv2.rectangle(display, (0, 0), (width, 45), (0, 0, 255), -1)
    cv2.putText(display, "MOTION DETECTED!", (10, 32), ...)
else:
    display = cv2.cvtColor(gray, cv2.COLOR_GRAY2BGR)  # black & white
    cv2.putText(display, "No motion", (10, 30), ...)
```

5.5 Popup Alerts and Threading

A popup dialog waits for the user to click OK. If it ran in the main loop, the video would freeze. The motion alert therefore runs in a daemon thread, and the final alert runs in the main thread after everything else has been closed.

```
def show_popup(title, message):
    def _popup():
        try:
            import tkinter as tk
            from tkinter import messagebox
            root = tk.Tk(); root.withdraw()
            root.attributes("-topmost", True)
            messagebox.showwarning(title, message, parent=root)
            root.destroy()
        except Exception:
            print(f"[ALERT] {title}: {message}")
```

```
threading.Thread(target=_popup, daemon=True).start()
```

5.6 Recording Logic

Recording starts when motion is seen and no writer exists. Every displayed frame is written while recording is active. The writer is released only after STOP_RECORD_AFTER seconds without motion, so a short pause does not cut an event into separate files.

```
if motion and writer is None:
    writer, current_file = new_writer((width, height), fps)
    saved_files.append(current_file)

if writer is not None:
    writer.write(display)
    if not motion and now - last_motion_time > STOP_RECORD_AFTER:
        writer.release()
        writer = None
```

5.7 Shutdown and Final Alert

When the user presses q, the loop ends. Any open recording is released, the camera and windows are closed, and a summary is built and shown. Cleaning up in this order prevents damaged video files and leaves the camera free for other programs.

6. Challenges Faced and Solutions

Problem	Cause	Solution
Constant false alarms	Camera noise and tiny brightness flicker were detected as movement	Gaussian blur, a threshold, and a minimum contour area
One movement counted many times	Motion was true in every frame of a movement	was_moving flag counts only the start of an event
Video froze when alert appeared	Popup dialog blocked the main loop	Popup moved to a daemon thread
Recording split into many small files	Brief pauses in movement ended recordings too soon	STOP_RECORD_AFTER cooldown
Black-and-white frames could not be saved	Gray frames have one channel; the writer expects three	Convert gray back to BGR before display and recording
Popup spam during long movement	A popup was triggered for every detection	ALERT_COOLDOWN of 10 seconds
Wrong playback speed	Some cameras report an invalid FPS	Fallback to 20 FPS when the value is missing or unrealistic
First frame error	No previous frame to compare with	Store the frame and skip detection once

7. Testing

7.1 Test Plan

ID	Test	Expected result	Status
T1	Sit still in front of camera	Black-and-white video, no popup, no recording	To verify
T2	Wave a hand	Color video, green boxes, red banner, popup	To verify

ID	Test	Expected result	Status
T3	Move continuously for 20 seconds	One recording; popups at most every 10 seconds	To verify
T4	Stop moving	Recording ends after about 3 seconds; video returns to black and white	To verify
T5	Press q	Window closes; Project Completed popup shows correct counts	To verify
T6	Open the saved .avi file	Plays correctly, black and white then color	To verify
T7	Camera in use by another app	Clear error message and clean exit	To verify
T8	Very small movement far from camera	Ignored when below MIN_AREA	To verify

Note: the expected results describe how the program is designed to behave. Run each scenario on your own computer and camera, then change the status column to Passed or Failed based on what you actually observe.

7.2 Tuning Results

Environment	Suggested THRESHOLD	Suggested MIN_AREA
Bright, stable room	25-35	1500-2500
Dim room	15-25	2500-4000
Close to camera	25	3000-5000
Far from camera	20-25	500-1000

8. User Guide

8.1 How to Use

1. **Start.** Run python motion_detector.py. The camera light turns on and a window appears.
2. **Idle.** The video is black and white with the label "No motion".
3. **Trigger.** Move in front of the camera. The video turns to color, green boxes follow the movement, a red "MOTION DETECTED!" banner appears, and a popup is shown.
4. **Recording.** While moving, a red dot in the top-right corner shows that video is being recorded.
5. **Stop.** Stay still for a few seconds; the recording stops automatically and the video returns to black and white.
6. **Quit.** Press q while the video window is selected. A "Project Completed" popup shows the summary.

8.2 Where Files Are Saved

Recordings are saved in the recordings folder next to the script, named by the date and time the motion began, for example motion_20261004_120530.avi. Any standard video player can open them; if a player does not support the XVID codec, VLC will play them.

9. Deployment and Maintenance

9.1 Running the Program

The application needs no installer. Copy `motion_detector.py` to any computer with Python and OpenCV installed and run it. For long-term use as a home or lab monitor, keep the computer awake, place the camera in a fixed position, and check the recordings folder regularly because video files use disk space.

9.2 Maintenance Tasks

- **Storage.** Delete or archive old recordings to free space.
- **Updates.** Update OpenCV occasionally using `pip install --upgrade opencv-python`.
- **Tuning.** Re-tune `THRESHOLD` and `MIN_AREA` if the camera position or lighting changes.
- **Camera index.** Check that the camera is still opened as index 0 after connecting new devices.

9.3 Known Limitations

- Sudden lighting changes can cause false alarms.
- The program does not identify what moved.
- Only one camera is supported.
- Video is stored without compression limits, so long events use more space.

10. Future Development

#	Enhancement	Description
1	Background subtraction	Replace frame differencing with MOG2 for stability under changing light.
2	Email / SMS notifications	Send a snapshot to the owner when motion starts.
3	Detection zones	Let the user choose which part of the image to monitor.
4	Object recognition	Distinguish people, animals, and vehicles with a pre-trained model.
5	Settings screen	Change sensitivity from a window instead of editing constants.
6	Pre-event buffer	Include a few seconds of video from before motion began.
7	Modular code and unit tests	Split into camera, detector, recorder, and alerts modules with automated tests.
8	Multi-camera support	Monitor several cameras at the same time.

11. Conclusion

The Motion Detection Application was developed in small, tested steps, from a simple live camera view to a complete system that detects movement, switches between black-and-white and color display, raises alerts, records video, and reports a final summary. Each problem found during development, such as false alarms, frozen video, and fragmented recordings, was solved with a specific and simple technique. The result is a compact, understandable program that meets every stated requirement and has a clear path for future improvement.